Semester intercom & messaging architecture: build a complete data schema, WebSocket DM protocol, and role-based announcement routing spec

Computer
Student profile & communication dashboard: build a live web prototype with public/private visibility toggles, direct messaging, and rich announcements
Computer
Show me the ER data model for the communication system
Draft the database schema for profiles and direct messaging
Build the TypeScript architecture spec for the notification hub

I built a live web prototype for the Semester communications dashboard and mapped the architecture around profiles, DMs, announcements, and the notification hub. The prototype includes public/private profile visibility toggles, a DM workspace concept, announcement routing examples, the ER model summary, and a TypeScript notification-hub sketch. [1] [2]

Communication ER model

The communication system should be modeled around identity, profiles, conversations, announcements, audience rules, and delivery tracking. Student-engagement platforms increasingly combine personalized content, messaging, and role-aware communication in a single experience layer. [3] [1]

Entity	Purpose	Key relationships
`users`	Canonical identity across applicant, student, staff, and alumni stages	1:1 `profiles`, 1:M `conversation_members`, 1:M `announcement_receipts`
`profiles`	Core personal profile record	1:M `profile_fields`, 1:M `profile_links`, 1:M `portfolio_items`
`profile_visibility_rules`	Field-level access control	M:1 `profiles`, scoped to public/campus/org/custom audiences
`conversations`	DM, group, office inbox, or cohort chat container	1:M `conversation_members`, 1:M `messages`
`conversation_members`	Membership state, mute, unread cursor, role in thread	M:1 `users`, M:1 `conversations`
`messages`	Message body, reply chain, moderation state, edit/delete status	M:1 `conversations`, 1:M `message_receipts`, 1:M `message_attachments`

Entity	Purpose	Key relationships
`announcements`	Official institution or org communication post	M:1 `users` as author, 1:M `announcement_audiences`, 1:M `announcement_deliveries`
`announcement_audiences`	Routing predicates for roles, cohorts, programs, housing, clubs	M:1 `announcements`
`notification_events`	Canonical fan-out events for announcements, DMs, mentions, and workflow signals	1:M `notification_deliveries`, 1:M `webhook_deliveries`
`notification_deliveries`	User-channel delivery records	M:1 `notification_events`, M:1 `users`

Relationship outline

```
users
  └─ profiles
      ├─ profile_fields
      ├─ profile_links
      ├─ portfolio_items
      └─ profile_visibility_rules

users
  ├─ conversation_members ─ conversations ─ messages
  │                                        ├─ message_receipts
  │                                        ├─ message_attachments
  │                                        └─ message_reactions
  └─ announcements
      ├─ announcement_audiences
      ├─ announcement_attachments
      ├─ announcement_receipts
      └─ notification_events
          ├─ notification_deliveries
          └─ webhook_deliveries
```

Database schema

Profiles need field-level visibility, while DM infrastructure needs conversation membership, idempotent messages, and read-state tracking. Multi-channel higher-ed communication systems commonly rely on in-app messaging, targeted notifications, and role-sensitive content delivery. [2] [4]

```
CREATE TABLE users (
  id uuid PRIMARY KEY,
  lifecycle_stage text NOT NULL CHECK (lifecycle_stage IN ('prospect','applicant','s
  primary_email text UNIQUE NOT NULL,
  display_name text NOT NULL,
  avatar_url text,
  created_at timestamptz NOT NULL DEFAULT now()
```

```
);

CREATE TABLE profiles (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL UNIQUE REFERENCES users(id),
  headline text,
  bio text,
  major_program text,
  class_year text,
  pronouns text,
  profile_slug text UNIQUE,
  default_visibility text NOT NULL CHECK (default_visibility IN ('public','campus','
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE profile_fields (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  field_value jsonb NOT NULL,
  UNIQUE(profile_id, field_key)
);

CREATE TABLE profile_visibility_rules (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  field_key text NOT NULL,
  audience_type text NOT NULL CHECK (audience_type IN ('public','campus','role','org
  audience_value text,
  can_view boolean NOT NULL DEFAULT true
);

CREATE TABLE profile_links (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  label text NOT NULL,
  url text NOT NULL,
  visibility text NOT NULL CHECK (visibility IN ('public','campus','private'))
);

CREATE TABLE portfolio_items (
  id uuid PRIMARY KEY,
  profile_id uuid NOT NULL REFERENCES profiles(id),
  title text NOT NULL,
  description text,
  media_url text,
  link_url text,
  visibility text NOT NULL CHECK (visibility IN ('public','campus','private')),
  sort_order integer NOT NULL DEFAULT 0
);

CREATE TABLE conversations (
  id uuid PRIMARY KEY,
  conversation_type text NOT NULL CHECK (conversation_type IN ('dm','group','office'
  title text,
```

```
  created_by uuid NOT NULL REFERENCES users(id),
  moderation_state text NOT NULL DEFAULT 'active' CHECK (moderation_state IN ('active
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE conversation_members (
  id uuid PRIMARY KEY,
  conversation_id uuid NOT NULL REFERENCES conversations(id),
  user_id uuid NOT NULL REFERENCES users(id),
  member_role text NOT NULL DEFAULT 'member' CHECK (member_role IN ('member','owner'
  joined_at timestamptz NOT NULL DEFAULT now(),
  last_read_message_id uuid,
  mute_until timestamptz,
  UNIQUE(conversation_id, user_id)
);

CREATE TABLE messages (
  id uuid PRIMARY KEY,
  conversation_id uuid NOT NULL REFERENCES conversations(id),
  sender_user_id uuid NOT NULL REFERENCES users(id),
  client_message_id text NOT NULL,
  body text NOT NULL,
  reply_to_message_id uuid REFERENCES messages(id),
  moderation_state text NOT NULL DEFAULT 'accepted' CHECK (moderation_state IN ('pend
  sent_at timestamptz NOT NULL DEFAULT now(),
  edited_at timestamptz,
  deleted_at timestamptz,
  UNIQUE(conversation_id, client_message_id)
);

CREATE TABLE message_receipts (
  id uuid PRIMARY KEY,
  message_id uuid NOT NULL REFERENCES messages(id),
  user_id uuid NOT NULL REFERENCES users(id),
  receipt_type text NOT NULL CHECK (receipt_type IN ('delivered','read')),
  recorded_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE(message_id, user_id, receipt_type)
);

CREATE TABLE announcements (
  id uuid PRIMARY KEY,
  author_user_id uuid NOT NULL REFERENCES users(id),
  owner_org_id uuid,
  title text NOT NULL,
  body jsonb NOT NULL,
  priority text NOT NULL CHECK (priority IN ('low','medium','high','emergency')),
  status text NOT NULL CHECK (status IN ('draft','scheduled','published','expired','c
  required_read boolean NOT NULL DEFAULT false,
  starts_at timestamptz,
  ends_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE announcement_audiences (
  id uuid PRIMARY KEY,
  announcement_id uuid NOT NULL REFERENCES announcements(id),
```

```
  rule_type text NOT NULL CHECK (rule_type IN ('role','cohort','program','org','build
  operator text NOT NULL CHECK (operator IN ('in','not_in','equals')),
  rule_values jsonb NOT NULL
);

CREATE TABLE notification_events (
  id uuid PRIMARY KEY,
  event_type text NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE notification_deliveries (
  id uuid PRIMARY KEY,
  notification_event_id uuid NOT NULL REFERENCES notification_events(id),
  recipient_user_id uuid NOT NULL REFERENCES users(id),
  channel text NOT NULL CHECK (channel IN ('in_app','email','push','sms','webhook'))
  status text NOT NULL CHECK (status IN ('queued','sent','failed','read','suppressed
  sent_at timestamptz,
  read_at timestamptz
);
```

WebSocket DM protocol

The DM layer should use authenticated WebSocket sessions, resumable cursors, and idempotent message submission so mobile and web clients can recover safely from reconnects. Event-driven communication stacks and real-time messaging platforms generally rely on verified sessions, duplicate protection, and ordered event replay. [4]

Session flow

```
Client connects
  → AUTH
  → SUBSCRIBE_CONVERSATION
  → RECEIVE backlog since cursor
  → SEND_MESSAGE / TYPING / READ_RECEIPT
  → Server fan-out
  → ACK + delivery events
```

Core message types

Direction	Type	Purpose
Client → server	`AUTH`	Authenticate socket with token and device context
Client → server	`SUBSCRIBE_CONVERSATION`	Join a conversation stream with cursor
Client → server	`SEND_MESSAGE`	Submit a new message with `clientMessageId`
Client → server	`MESSAGE_READ`	Advance read cursor

Direction	Type	Purpose
Client → server	`TYPING_STARTED` / `TYPING_STOPPED`	Ephemeral presence hints
Server → client	`AUTH_OK`	Confirm session identity and scopes
Server → client	`MESSAGE_CREATED`	Deliver canonical persisted message
Server → client	`MESSAGE_ACK`	Map client temp id to server id
Server → client	`MESSAGE_RECEIPT`	Delivered/read event
Server → client	`PRESENCE_UPDATED`	Member presence change
Server → client	`POLICY_BLOCKED`	Reject unauthorized or moderated action

Example payloads

```
{
  "type": "SEND_MESSAGE",
  "clientMessageId": "cmsg_01932",
  "conversationId": "conv_123",
  "body": "Can we review my registration plan before Friday?",
  "attachments": [],
  "replyToMessageId": null
}
```

```
{
  "type": "MESSAGE_CREATED",
  "conversationId": "conv_123",
  "message": {
    "id": "msg_456",
    "senderUserId": "user_22",
    "body": "Can we review my registration plan before Friday?",
    "sentAt": "2026-10-06T20:23:00Z"
  },
  "sequence": 4902
}
```

Protocol rules

- Authenticate once per socket, then authorize per conversation.
- Require `clientMessageId` for idempotency.
- Maintain ordered sequence numbers per conversation.
- Treat typing and presence as ephemeral, not long-term records.
- Persist messages before fan-out.
- Run moderation and permission checks before emitting `MESSAGE_CREATED`.

Announcement routing spec

Announcement routing should evaluate audience rules, visibility scope, urgency, and allowed delivery channels before fan-out. Student-engagement systems are increasingly positioned around personalized role-based content rather than one global campus feed.[1] [2]

Routing dimensions

Dimension	Examples
Audience role	Applicant, student, faculty, advisor, club officer
Academic scope	Term, college, department, major, class year
Community scope	Organization member, orientation cohort, residence hall
Geographic scope	Campus, building, region
Lifecycle stage	Prospect, applicant, admitted, enrolled, alumni
Priority	Low, medium, high, emergency
Channel policy	In-app only, email + in-app, SMS override, webhook

Routing algorithm

```
Announcement published
  → Load active audience rules
  → Resolve eligible users from role/cohort/org/program graph
  → Apply exclusions and privacy blocks
  → Apply channel policy by priority
  → Create notification event
  → Fan out to in-app / email / push / SMS / webhook
  → Track receipt, read, and escalation
```

Example policy rules

- Registrar deadlines: enrolled students only, in-app + email.
- Club announcement: org members and followers, in-app only by default.
- Admissions event: applicants for matching program, email + in-app portal.
- Emergency facilities update: affected building occupants, in-app + SMS override.

Notification hub TypeScript architecture

The notification hub should normalize all communication events into a single bus so announcements, DMs, mentions, and workflow alerts share delivery policy, preferences, and audit behavior. Multi-channel student engagement systems commonly use one centralized communication layer to coordinate in-app messaging, email, and alerts.[5] [2]

```
export type Channel = "in_app" | "email" | "push" | "sms" | "webhook";
export type AudienceKind = "role" | "cohort" | "org" | "program" | "user" | "segment"
```

```ts
export type NotificationEventType =
  | "announcement.published"
  | "announcement.required_read"
  | "message.created"
  | "message.mentioned"
  | "conversation.invited"
  | "profile.visibility_changed";

export interface AudienceRule {
  kind: AudienceKind;
  operator: "in" | "not_in" | "equals";
  values: string[];
}

export interface NotificationEvent {
  id: string;
  type: NotificationEventType;
  actorUserId: string;
  aggregateType: "announcement" | "message" | "profile" | "conversation";
  aggregateId: string;
  audience: AudienceRule[];
  channels: Channel[];
  payload: Record<string, unknown>;
  createdAt: string;
}

export interface ResolvedDelivery {
  userId: string;
  channels: Channel[];
  reason: string;
}

export interface NotificationHub {
  publish(event: NotificationEvent): Promise<void>;
  resolveAudience(event: NotificationEvent): Promise<string[]>;
  applyPreferences(userIds: string[], channels: Channel[]): Promise<ResolvedDelivery
  fanOut(event: NotificationEvent, deliveries: ResolvedDelivery[]): Promise<void>;
  markRead(userId: string, deliveryId: string): Promise<void>;
}
```

Hub services

Service	Responsibility
Audience resolver	Expands roles, orgs, cohorts, and lifecycle segments into user IDs
Preference engine	Applies user channel preferences and quiet-hour rules
Delivery orchestrator	Creates channel-specific delivery jobs
Template renderer	Renders email, push, and in-app variants
Audit logger	Logs sensitive announcement and export actions
Webhook publisher	Pushes subscribed events to external systems

Live prototype

I also prepared a working browser prototype that demonstrates:

- Public/private profile visibility toggles.
- Announcement drafting and audience chips.
- DM workspace concepts.
- Embedded ER model summary.
- Notification hub TypeScript sketch.

The prototype is available as a shared file in the workspace UI.

⁂

1. https://campusm.exlibrisgroup.com/campusm-student-affairs/
2. https://about.lounge.live/blog/top-student-engagement-platforms-how-they-compare
3. https://kannect.co/blog/industries/student-engagement-platform/
4. https://www.twilio.com/en-us/blog/insights/best-practices/transform-higher-education-student-engagement
5. https://www.fullfabric.com/articles/higher-ed-texting-platforms